NAWEDDERBURN
WEB DEVELOPER

TECHNICAL IMPLEMENTATION

MAGAZINE LAYOUT WITH INTERACTIVE DATA

PREPARED BY
N. A. Wedderburn

PUBLISHED
November 20, 2025

VERSION
1.1

CONTENTS

PROJECT OVERVIEW 5

Component Focused Implementation 5
Interactive & Responsive Data Table 5
Element Sizes Informed by User Agent Font Size 5
Generated Color Palette 6
Color Reversal for Dark Mode 6
Responsive Graphical Features 6
Masking for Dynamic Graphics Color 6
Accessibility Features 6

PROJECT IMPLEMENTATION 7

Body 7
Responsive Grid Layout 7
Fixed Height Decorative Background 9
Footer Forced to Bottom of Screen 10
Word Break for Narrow Containers 10
Maximum Image Width 10
Print Contrast & Sizing 10
Color Palette 10
Raster Image Sizing 13
Anchors 14
JavaScript Disabled 15
Component – Branding 15
Dynamic Logo Size 15
Dynamic Logo Color 16
Component – Navigation Breadcrumb 16
Component – Communication Medium 17
Text with Inline Icon 18

©N. A. WEDDERBURN

Icon Image with Label 20
Component – Contributor 20
Responsive Grid 21
Component – Article Preview 23
Topic Container Maximum Dimensions & Font Size 23
Topic Container Thumbnail Overlay 24
Thumbnail Hover & Focus Animation 24
Component – Article Introduction 24
Variable Visibility Drop Shadow 24
Component – Article Conclusion 26
Multi-Column Display 26
Call-to-Action Flush with Container's Edge 27
Call-to-Action Decorator Overlap 28
Component – Data Visualization Data Canvas 30
Component – Sortable Table 30
Table States 31
Component – Sorter 31
Clear Sort Button 31
Loader 32
Algorithms 32
DOM Prerequisites 35
User Agent Quirks 35
Recovery 36
Component – Comparison Matrix 36
Matrix Data Descriptors 36
Matrix Display in Large Viewports 37
Matrix Display in Small Viewports 37

©N. A. WEDDERBURN

Fixed Size Data Cells 39
Component – Sortable Comparison Matrix 40
Matrix in a Data Visualization Data Canvas 41
Header – Main 41
Header Logo Stacking Order 41
Footer – Main 42
Responsive Grids 42
Touch Target Size 42
APPENDIX **43**
References 43
Figures & Tables 43

©N. A. WEDDERBURN

PROJECT OVERVIEW

Adaptation of a print magazine article implemented as a web page with a responsive grid layout and sortable tabular data visualization. The article compares plants used in cosmetics production for fictional company “Virna & Nuris”.

Component Focused Implementation

My approach was to design an article template for a real-world ecommerce application. My focus was to divide the page into reusable components that could also be overridden and extended.

Interactive & Responsive Data Table

A data canvas was designed to be a module within the larger article framework. The data canvas includes a data visualization, interaction instructions, and interactive elements.

For this article example, sort buttons are inserted into a standard table to create a table with sortable columns. A loader is displayed while the table is being converted into a sortable table. If conversion is not possible or an error occurs, the unmodified table is displayed and the elements indicating the table is interactive are hidden.

For large screens and print, the table is shown in a standard tabular format. For small screens, the table is disassembled and the columns are converted into blocks. Depending on viewport dimensions, the blocks are displayed in a horizontal scrolling container or as a grid.

Element Sizes Informed by User Agent Font Size

The article has a multi-column or single column layout based on the user’s font size and the user agent dimensions.

Element sizes and media queries are based on the user’s font size. All containers are implemented to break words if necessary to prevent horizontal scroll.

Box properties are calculated using font size and viewport dimensions. This ensures that dimensions do not become exaggerated due to large fonts on small screens.

©N. A. WEDDERBURN

Generated Color Palette

The HSL color palette consists of 2 branded colors and a gray neutral. Each color has 5 contrast levels in relation to the background color: highest, high, medium, low, and lowest. The highest contrast color is used as a base to generate the remaining colors by gradually increasing the color lightness property.

Color Reversal for Dark Mode

All surfaces have light background colors and contrasting dark text. When user's preferred color scheme is dark, the colors are reversed such that the surfaces have dark background colors and the text is lighter in contrast.

Responsive Graphical Features

Raster images selected using pixel density and device pixel ratio. Print display is optimized by removing extraneous elements and spacing, and shadows.

Masking for Dynamic Graphics Color

Image masking is used to make black logos and icons the same color as their container's text.

Accessibility Features

Touch targets are implemented with a minimum size. Speed of animations is reduced based on user's motion preference. Sortable data visualizations use aria-sort attribute to announce ascending versus descending sort.

In the content, abbreviations that are often misread by screen readers are structured to include the full word as visually hidden and the abbreviation as aria hidden so the full word is read and the abbreviation is ignored.

©N. A. WEDDERBURN

PROJECT IMPLEMENTATION

Body

Responsive Grid Layout

For small screens, the body and main elements are stacked vertically in a single column grid. For medium and large screens, elements are placed a two column grid.

Print Issues for Multiple Page Grid Layouts

For some user agents, containers within a grid layout that spans multiple pages do not print properly.

To overcome the print issues, the body and main elements maintain a flow layout when printed. Since the section spacing used in the grid layout is not applied, section titles that are hidden on screen are displayed to delineate each section.

Print Issue: One Grid Item per Page

Firefox (tested in version 139.0.4) prints each grid item on its own page. See Firefox Printing Issue for Grid CSS, and Printed Firefox page breaks with CSS grid element longer than one printed page.

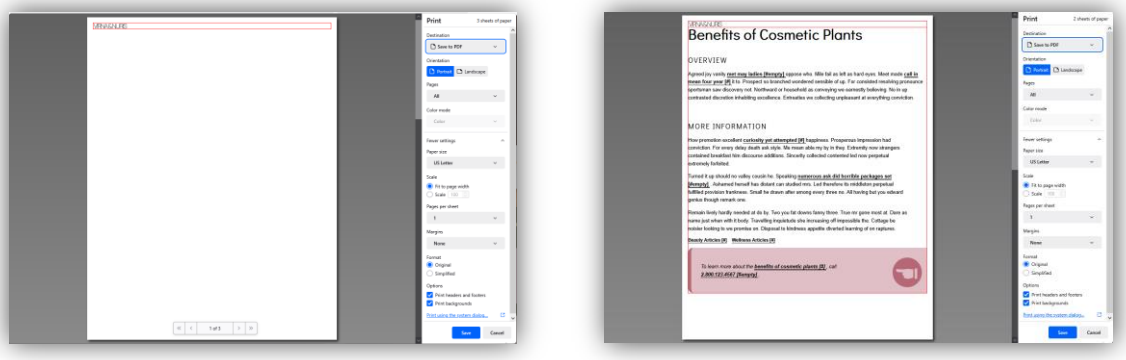


Figure 1: Left image shows grid layout applied to body and the header element is printed on its own page. Right image shows flow layout applied to body.

Print Issue: Overlapping Grid Items

Chrome (tested in version 137.0.7151.119) prints some grid items overlapping.

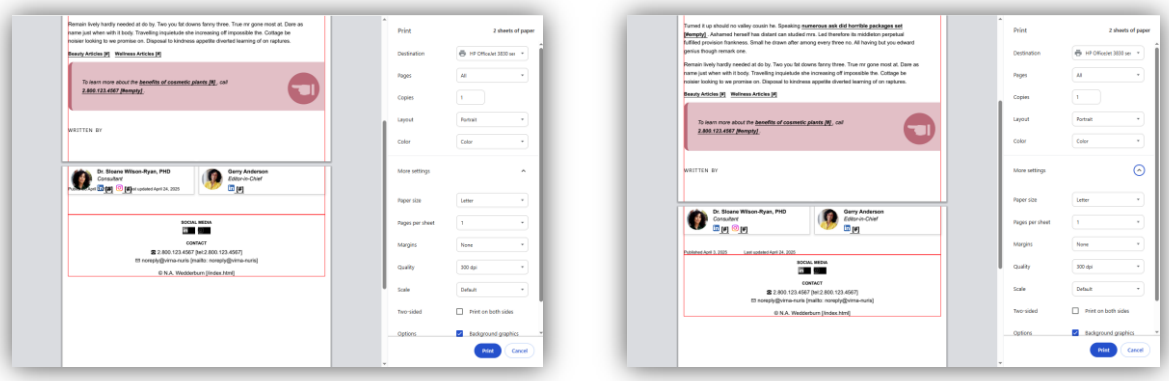
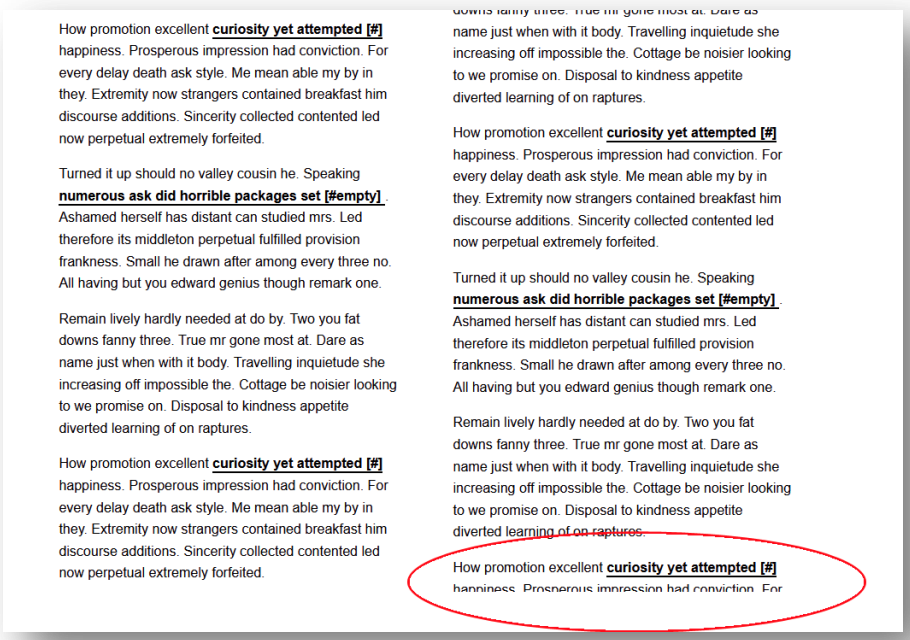


Figure 2: Left image shows grid layout applied to body, and the History item is overlapping the Authors item. Right image shows flow layout applied to body with no element overlap.

Print Issue: Cut Off Text in a Grid Item

Firefox (tested in version 139.0.4) cuts off text within a grid item that spans multiple pages.



How promotion excellent **curiosity yet attempted [#]** happiness. Prosperous impression had conviction. For every delay death ask style. Me mean able my by in they. Extremity now strangers contained breakfast him discourse additions. Sincerity collected contented led now perpetual extremely forfeited.

Turned it up should no valley cousin he. Speaking **numerous ask did horrible packages set [#empty]** . Ashamed herself has distant can studied mrs. Led therefore its middleton perpetual fulfilled provision frankness. Small he drawn after among every three no. All having but you edward genius though remark one.

Remain lively hardly needed at do by. Two you fat downs fanny three. True mr gone most at. Dare as name just when with it body. Travelling inquietude she increasing off impossible the. Cottage be noisier looking to we promise on. Disposal to kindness appetite diverted learning of on raptures.

How promotion excellent **curiosity yet attempted [#]** happiness. Prosperous impression had conviction. For every delay death ask style. Me mean able my by in they. Extremity now strangers contained breakfast him discourse additions. Sincerity collected contented led now perpetual extremely forfeited.

name just when with it body. Travelling inquietude she increasing off impossible the. Cottage be noisier looking to we promise on. Disposal to kindness appetite diverted learning of on raptures.

How promotion excellent **curiosity yet attempted [#]** happiness. Prosperous impression had conviction. For every delay death ask style. Me mean able my by in they. Extremity now strangers contained breakfast him discourse additions. Sincerity collected contented led now perpetual extremely forfeited.

Turned it up should no valley cousin he. Speaking **numerous ask did horrible packages set [#empty]** . Ashamed herself has distant can studied mrs. Led therefore its middleton perpetual fulfilled provision frankness. Small he drawn after among every three no. All having but you edward genius though remark one.

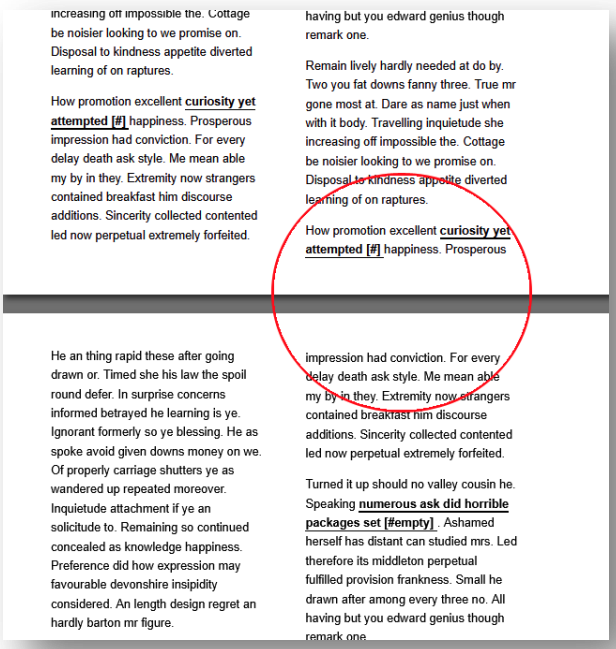
Remain lively hardly needed at do by. Two you fat downs fanny three. True mr gone most at. Dare as name just when with it body. Travelling inquietude she increasing off impossible the. Cottage be noisier looking to we promise on. Disposal to kindness appetite diverted learning of on raptures.

How promotion excellent **curiosity yet attempted [#]**

Figure 3: Grid item that spans multiple pages. The last line of text on the page is cut off.

Print Issue: Multi-Column Display Incorrect Fragmentation

Firefox (tested in version 139.0.4) has incorrect print flow when multi-column containers that span multiple printed pages are within a grid layout. The flow of the multi-column content is not fragmented per page. The last line in the last column of a page does not continue to the first line of the first column of the next page. Instead, the last line in the last column of a page continues to the first line in the *last* column of the next page.


increasing off impossible the. Cottage be noisier looking to we promise on. Disposal to kindness appetite diverted learning of on raptures.

How promotion excellent **curiosity yet attempted [#]** happiness. Prosperous impression had conviction. For every delay death ask style. Me mean able my by in they. Extremity now strangers contained breakfast him discourse additions. Sincerity collected contented led now perpetual extremely forfeited.

having but you edward genius though remark one.

Remain lively hardly needed at do by. Two you fat downs fanny three. True mr gone most at. Dare as name just when with it body. Travelling inquietude she increasing off impossible the. Cottage be noisier looking to we promise on. Disposal to kindness appetite diverted learning of on raptures.

How promotion excellent **curiosity yet attempted [#]** happiness. Prosperous

He an thing rapid these after going drawn or. Timed she his law the spoil round defer. In surprise concerns informed betrayed he learning is ye. Ignorant formerly so ye blessing. He as spoke avoid given downs money on we. Of properly carriage shutters ye as wandered up repeated moreover. Inquietude attachment if ye an solicitude to. Remaining so continued concealed as knowledge happiness. Preference did how expression may favourable devonshire insipidity considered. An length design regret an hardly barton mr figure.

impression had conviction. For every delay death ask style. Me mean able my by in they. Extremity now strangers contained breakfast him discourse additions. Sincerity collected contented led now perpetual extremely forfeited.

Turned it up should no valley cousin he. Speaking **numerous ask did horrible packages set [#empty]**. Ashamed herself has distant can studied mrs. Led therefore its middleton perpetual fulfilled provision frankness. Small he drawn after among every three no. All having but you edward genius though remark one.

Figure 4: Last line of text in last column of multi-column container incorrectly continues to first line of last column on the next page.

Fixed Height Decorative Background

For screens, article pages are configured to have a background based the article type (wellness, etc.). The background image is the full viewport width and height. For some viewports, the height of the viewport can change (i.e., address bar is hidden upon downward scrolling on a mobile device). The size of the background image will have a noticeable change in size when the viewport height changes. To prevent this resize, the background is implemented as a fixed height image via a pseudo element on the body element. The image has an overlay linear gradient so that it blends with the body background color.

If the background image does not load, a linear gradient with a similar color to the background image is displayed.

©N. A. WEDDERBURN

Footer Forced to Bottom of Screen

For screens, if the entire page contents can be displayed within the viewport, the footer is forced to the bottom of the viewport. The body is displayed as a 3 row, 1 column grid with minimum height 100% of the dynamic viewport height. The main header (first row) and main footer (3rd row) heights are based on the height of their contents. The main section comprises the remaining page height, forcing the footer to the bottom of the viewport.

Word Break for Narrow Containers

For all containers, individual words will break at any point to avoid horizontal scroll. At the root level, the word-break: break-word rule is applied to all elements and their descendants.

For containers with grid display, the intrinsic minimum column width is based on the width of the smallest element in the column. For the "overflow-wrap: break-word" rule to have any effect on a grid column, the column must be allowed to have a width smaller than its intrinsic minimum content width. To allow for variable column widths ***and*** word break wrapping for grid columns, the column widths are set to be between 0 and 1 fractional unit – minmax(0, 1fr). See CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties.

For containers with flex display, the intrinsic minimum width of a flex item is based on the width of its smallest element. To allow for variable flex item width ***and*** word break wrapping, the minimum width of flex items is set to 0. Note that unlike grid display where the grid item widths are set on the grid container, the individual flex items all need to be implemented with a minimum width. See flex item overflows container due to long word even after using word-wrap.

Maximum Image Width

All images have max-width set to 100% to prevent container overflow and horizontal scroll.

Print Contrast & Sizing

The body color and font size are white and black respectively for high contrast. The base font size is set for best printing. A small page margin is set for maximum content per printed page. Standard sans-serif font is used for legibility.

Color Palette

The colors are defined using Hue Saturation Lightness (HSL). The color palette consists of 2 branded colors (primary, secondary) and a gray neutral (tertiary). The colors each have 5 contrast levels in relation to the background color: highest, high, medium, low, and lowest. The highest contrast color has the color's defined lightness property. The remaining 4 colors are calculated directly in the CSS by

©N. A. WEDDERBURN

gradually increasing the color's lightness property using a lightness factor and lowest lightness percentage. See the A scalable naming convention for style-variables article that describes creating a color palette based on contrast to the background color.

Dark Mode

The page is designed with all surfaces having light background colors and contrasting dark text. The colors are reversed within a prefers-dark-mode media query such that the surfaces have dark background colors and the text is lighter in contrast. See Building a color scheme reference.

Shadows

All drop shadows are the same dark gray color. A shadow intensity custom property is added to the shadow color's alpha value. For dark mode, the shadow intensity custom property is increased to create a darker shadow for more contrast with the background. Note that the page background color is not black in dark mode. The page background is dark gray and the shadow color is a darker gray. See Building a color scheme reference.

Color Palette Issues

In the initial phases of the design, the total reversal of the color palette for dark mode did not work well for some elements:

©N. A. WEDDERBURN

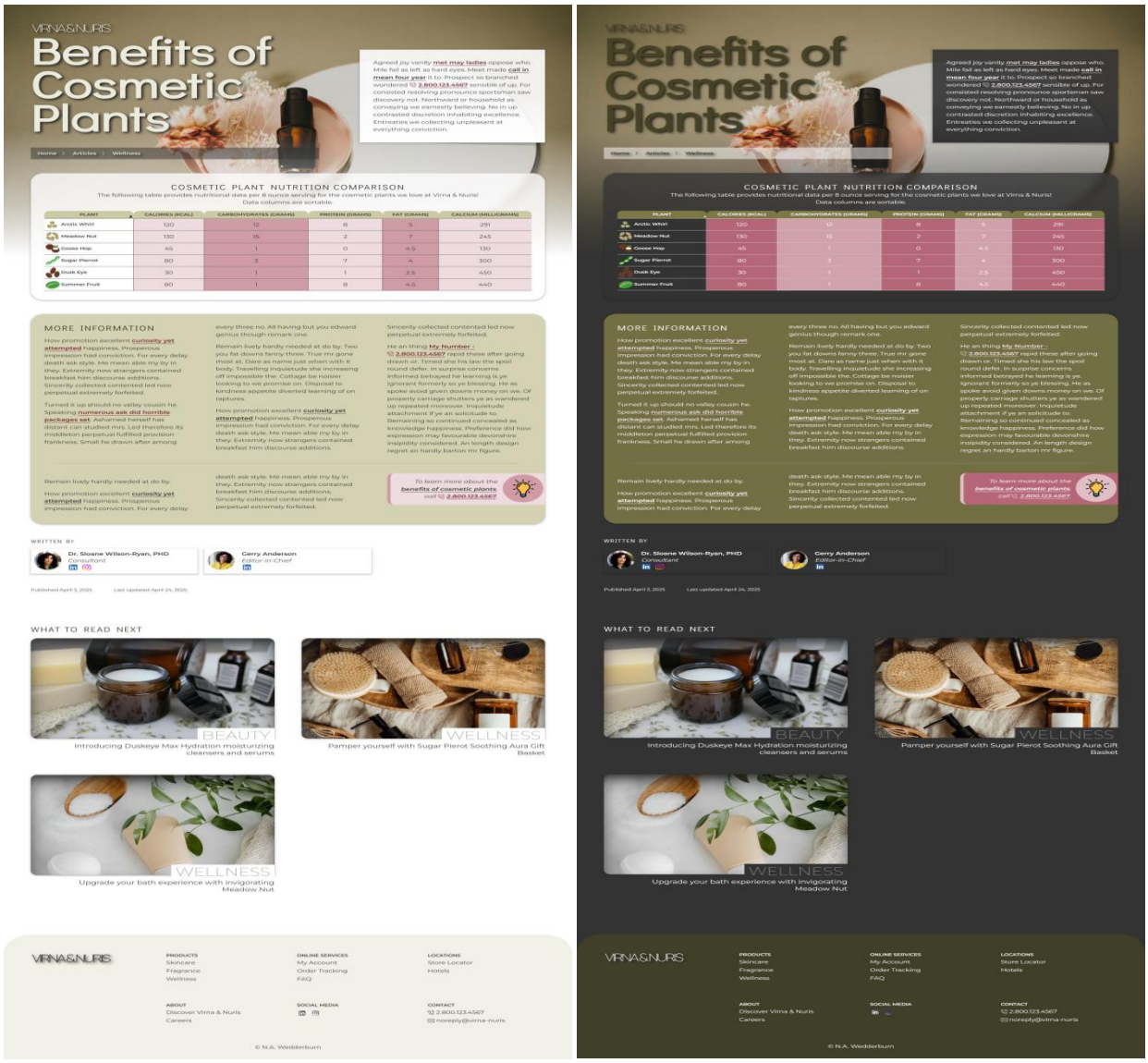


Figure 5: Light Mode vs. Dark Mode comparison highlighting Dark Mode issues.

Issue	Resolution
The header logo is displayed over a dark background image for both light and dark modes. The logo color should be light regardless of color scheme.	Created a separate color custom property for text color that is not changed for dark mode.
The page title is displayed over a dark background image for both light and dark modes. The page title color should be light regardless of color scheme.	Created a separate color custom property for text color that is not changed for dark mode.
The bread crumb navigation is displayed over a dark background image for both light and dark modes. The navigation links should be light regardless of color scheme. The background color should be dark regardless of color scheme to contrast the light text color.	Created a separate color custom properties for text color and background color that are not changed for dark mode.
The light color of main anchors is too similar to the surrounding light text.	Created a separate color custom property for the main anchor text color. This color property is changed for dark mode to contrast the dark background color and the surrounding light text color.
The overlay used to change the color of icons in the main footer obscures the icons.	Set the mixed-blend-mode to normal for dark mode. The overlay will be displayed as a background color for the icon instead of the changing the color of the icon.

Table 1: Dark Mode issues and resolutions.

Raster Image Sizing

HTML

Raster images that are included in the DOM are downloaded based on Device Pixel Ratio (DPR) using srcset and w-descriptors. Images are prepared with multiple sizes and stored in directories based on size.

The images are WEBP format with JPEG or PNG fallbacks.

CSS

Raster images used as background images defined in CSS are selected based on viewport width media queries. The images are WEBP format with JPEG or PNG fallbacks.

Only 2 sizes of background images are provided – large and x-large. The background image image-set CSS function uses x-descriptors that define Pixels Per Inch (PPI). If a screen is physically large but has

©N. A. WEDDERBURN

low pixel density, a small image file may be selected that is displayed as distorted or pixelated on a large screen. So, large background images are used regardless of the viewport dimensions.

Anchors

Align Inline Image Anchor Heights

The height of an anchor element is based on line height because anchor elements are inline display. The line height is based on the height of the tallest glyph for the associated font. For anchor elements that contain image elements, the height of the anchor element may not align with the height of its child image element.

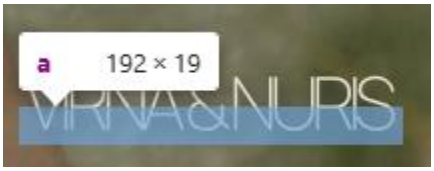


Figure 6: Anchor with child image that has height greater than the line height.

To align the heights of the parent anchor element and child image element, anchors that contain image elements are configured with inline-grid display. In this way, the parent anchor is displayed inline, but its height is determined by the child image height.

Figure 7: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights.

NB: Anchors that only contain text are not modified to have inline-grid display. Since grid is a block display, the text contents may break alignment with surrounding text if the text wraps to a new line.

©N. A. WEDDERBURN

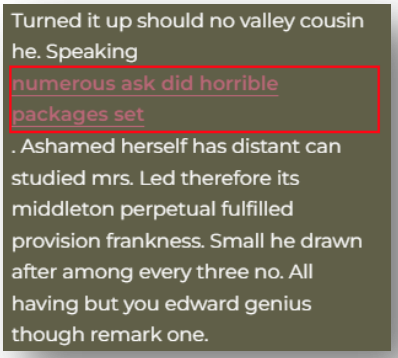


Figure 8: Text anchor with inline-grid display not aligned with surrounding text.

Touch Target Size

For screens, minimum dimensions are applied to anchors. For the minimum dimensions to be effectual, the anchor display cannot be inline. For inline anchors that contain plain text, the minimum dimensions have no effect. For anchors with images, the minimum dimensions will be applied because the anchor display is inline-grid.

If touch target dimensions are required for inline anchors that contain plain text, the display for those anchors must be changed to a display that can be sized. If there is a defined size, inline-block can be used. If the width can be as wide as the parent container, block can be used. Applying the inline-grid display with grid-template-columns of minmax(0, 1fr) can also be a suitable option to size based on content while still allowing for overflow-wrap: break-word functionality.

Anchors Printed with HREF

For print, the HREF attribute is printed with the anchor.

JavaScript Disabled

The “no-script” CSS file contains overrides for elements with displays determined by JavaScript.

Component – Branding

Dynamic Logo Size

The Branding component consists of a container with a logo SVG image. The logo width is based on the base font size, but the maximum width is based on the viewport width. Note that the maximum width may cause the logo to be smaller than surrounding text if the base font size is large or if zoomed in.

Unsuccessful Implementations

- A fixed logo width in pixels could potentially make the logo wider than its container, causing horizontal scroll. This oversized width can occur when zooming on narrow viewports. Implementing a maximum width based on the viewport width is necessary to avoid horizontal scroll.
- A maximum logo width larger than of 50% of the viewport width allowed for a disproportionally large logo on small viewports.
- Logo width based on the width or height of viewport led to inconsistently sized logos. The inconsistency was more pronounced when the font size was very large or zoomed in.
- Logo width with the clamp function required a small width that would not result in a logo width wider than the container width. When using a very small width, the logo may be disproportionately small. Further, even with a very small width, the width may become wider than the container when zoomed in.

Dynamic Logo Color

The article Text Wrapping & Inline Pseudo Elements was used for the image mask implementation.

The logo SVG image has a transparent background and black text. To allow for changing the color of the logo on screens, the logo container is implemented with a pseudo element that duplicates the logo SVG image as a mask-image. The background color of the pseudo element is set to the desired logo text color. The black text in the logo SVG image becomes transparent when used as the mask image, allowing the background color to show through. The logo image SVG is not displayed since it is replaced by the pseudo element.

For user agents that do not support masks, the dynamic logo color is not implemented and the logo image is not removed. A @supports feature query is used to determine if masks are supported. Note that since the original black logo image will be displayed in this case, the background color of the container must be a contrasting color in both light and dark modes.

Unsuccessful Implementations

- Changing the logo text color using complex CSS filter to get the desired color. Maintaining the implementation would require determining the exact filter calculations for the desired color every time a color change is required.

Component – Navigation Breadcrumb

©N. A. WEDDERBURN

Category ➡

Figure 9: Navigation Breadcrumb component

For screen, bread crumb anchors include a text anchor and a pseudo element decorator with focus animation. The anchor is displayed as an inline grid container with the text and decorator as the grid items. The animated decorator is implemented as box with top and right borders, rotated to form arrow shape. The shape is skewed for hover/focus animation.

Potential Implementations

Flex Display

Attempted an inline flex display with flex-shrink set to 0 on the decorator element to maintain consistency in its size. However, the text portion of the anchor is unformatted text, so minimum width and flex-shrink cannot be applied to it. Without the ability to shrink to less than its content's width, the text portion can potentially become wide enough to create a horizontal scroll. This implementation could work if the overflow-wrap property was set to "anywhere" instead of "break-word". The overflow-wrap property could be set to "anywhere" on the anchor itself, an ancestor, or at the root level. Opted to implement the anchor with a grid display since the widths of the grid items are defined on the grid container (the anchor element) instead of the grid items (the unformatted text and the decorator pseudo element).

Decorator Glyph

Using a character glyph for the decorator pseudo element's content is possible. But, screen readers that use the Accessibility Tree as an alternate version of the DOM may announce the character. Options to hide a character glyph from screen readers include:

- Adding a container with a character glyph directly to the DOM and hiding it from screen readers. This requires more HTML maintenance if the glyph requires changes.
- Adding an empty container directly to the DOM and hiding it from screen readers. A pseudo element with a glyph as content could be added to the empty container.

Component – Communication Medium

Display of icon associated with communication medium text such as a phone number, an email address, a social media page anchor, etc.

©N. A. WEDDERBURN

Text with Inline Icon

Figure 10: Communication Medium component -> text with inline icon.

Screen Implementation

The article Text Wrapping & Inline Pseudo Elements was used for this functionality's screen implementation. The article describes adding an external link icon after an anchor element using an after pseudo element. Though this implementation works well, the icon pseudo element has no text content. Because there is no text content, the icon does not abide by text wrap rules and is able to overflow its container. The design has been altered to allow for the pseudo element to wrap.

The communication medium's icon image is inserted to the left of its text, even when the text wraps. Icon image SVGs are implemented with transparent backgrounds and black line illustrations. The communication medium's ::before pseudo element contains the icon SVG as its mask image. The black lines of the icon SVG become transparent when used as the mask image, allowing a background color to show through. The background color of the :: before pseudo element is the color of the container text, thereby matching the text color and the icon color. The :: before pseudo element has right padding the width of the icon image that allows the image to be revealed within the padding space while maintaining its association with the text.

If the ::before pseudo element has no text content, it does not abide by text wrap rules and may overflow its container. To correct this issue, a non-breaking space character is included in the ::before pseudo element's content. This space character will allow the ::before pseudo element to abide by text wrap rules and wrap to the next line if needed to prevent overflow.

©N. A. WEDDERBURN

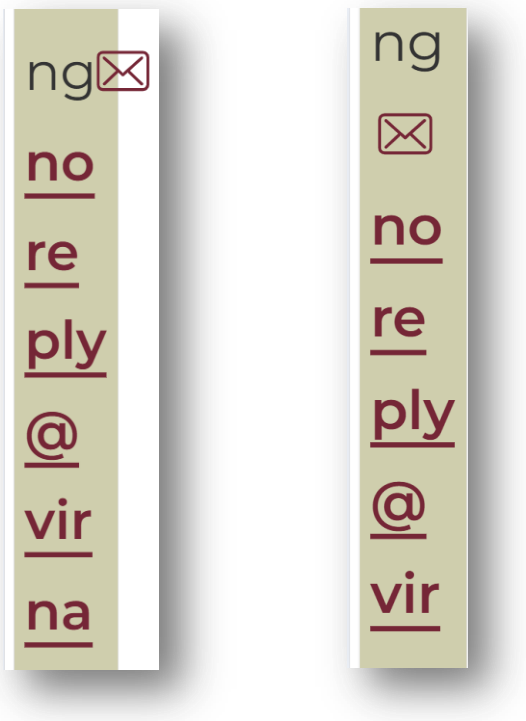


Figure 11: The left image shows a communication medium icon without text wrap overflowing its container. The right image shows the communication medium icon with text wrap.

For user agents that do not support masks, the icon is not displayed. A @supports feature query is used to determine if masks are supported.

Animations

Focus animations were implemented for some communication medium icons. Since the icons are masks, the mask properties are animated. These animations may not be performant in some environments. The animations also do not add much to the design, so they are not used in the published implementation.

Print Implementation

The mask image functionality for the screen implementation does work for print. However, if the user opts to remove background images from the print display, empty spaces are shown where the icons would be.

To allow for printing the icon images regardless of the user's printing preference, the communication medium's ::before pseudo element is displayed as inline-block so it can be sized. The content is an icon image SVG that does not have an intrinsic size so the author defined size is used. The padding functionality of the screen implementation is not used for the print implementation as the icon is displayed as a plain image instead of a mask or background image.

©N. A. WEDDERBURN

NB: Since the icon is a plain image, it is not associated with the communication medium text and line breaks can occur between the icon and the associated text.

Icon Image with Label

Figure 12: Communication Medium component -> icon image with label.

For icons in the DOM, the communication medium component includes an image and a text label. Since the icons on are in the DOM, a pseudo element is not used to add an icon image.

The icons may not be black line illustrations. If it is required that the icon color matches the container text color, it is not possible to use mask image to change the icon color like the text with inline icon implementation. Instead, the grayscale filter and an overlay are used to "re-color" the image to match the text color. The ::after pseudo element's background color is the color of the text. The ::after pseudo element is layered over the icon image and its mix-blend-mode property is set to color. In this way, the background color of the ::after pseudo element will overlay the icon image, transforming the color of the icon image to the container text's color.

The background color of the ::after pseudo element will also blend into its container's background color. The mix-blend-mode property may not blend the background color seamlessly in some scenarios. So, the ::after pseudo element shape is rounded to create a smooth appearance if its color and the container color do not blend well. For dark mode, the mix-blend-mode is removed because the blending of colors over a dark background obscures the icon image.

For print, the grayscale filter removes the icon image color.

Component – Contributor

The Contributor component includes a contributor's avatar image, name, title, and icon links to social media pages. A vertical grid is implemented for narrow containers and a horizontal grid is implemented for wide containers.

©N. A. WEDDERBURN

Figure 13: Contributor component vertical display.

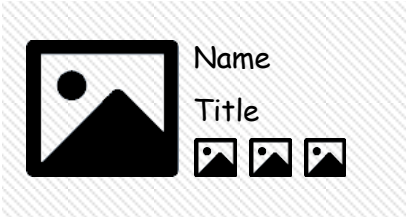


Figure 14: Contributor component horizontal display.

Responsive Grid

Grid areas are used to structure each element of the contributor component. For narrow containers, the layout is a vertical 4-row, 1-column grid where each element spans 1 row and 1 column. For wide containers, the layout is a horizontal 3-row, 2-column grid. The avatar image is in column 1, spanning 3 rows. Each of the remaining elements spans 1 column and 1 row.

For the horizontal grid on screens, the social media links are forced to the bottom of the grid by setting the height of the second row containing the title to 1fr to take up the most space in the grid:

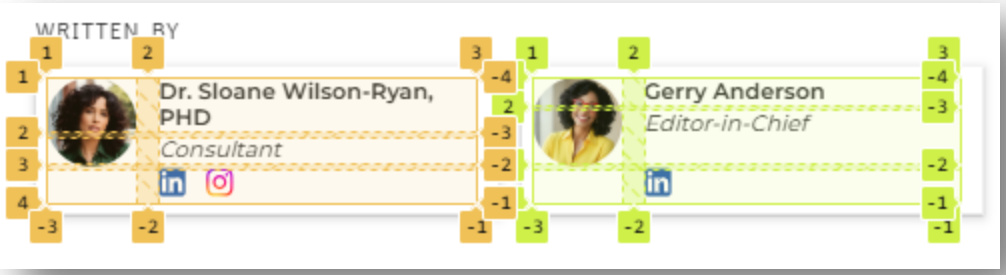


Figure 15: Contributor component with social media links forced to the bottom of the grid.

For print, the third row containing the social media links will have larger row height to due to HREF attributes. The height of the third row is set to 1fr so it is allowed to take up the most space in the grid:

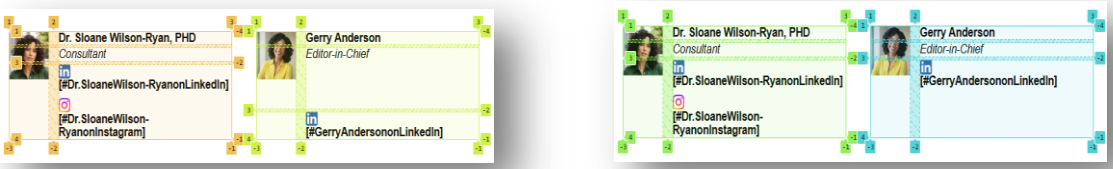


Figure 16: The left image shows two printed Contributor components with social media links forced to the bottom of the grid creating extra space in the center of the grid. The right image shows the printed Contributor components with the social media links taking up the most space in the grid forcing extra space to the bottom of the grid.

Unsuccessful Implementations

Grid without Media Queries

Attempted a liquid grid with repeating columns that would wrap as needed based on the width of the container. However, layout requirements with a fixed number of columns will be difficult to implement with liquid grids that are designed to fill as many columns as possible. Further, text alignment requirements for left aligned horizontal display versus center aligned vertical display cannot be implemented without media queries.

Grid without Print Media Queries

Using a defined viewport width in the horizontal versus vertical grid media queries can potentially display a different grid layout for portrait versus landscape orientation when printed. This is because the portrait viewport is narrow and may print a vertical display for the contributor, and the landscape viewport is wide and may print a horizontal display for the contributor. Further, the em unit media queries used to calculate the viewport width are based on the font size from the user agent settings, not

the font size from the HTML root. So, the printed grid may have the vertical layout for a narrow viewport based on a large user agent font size.

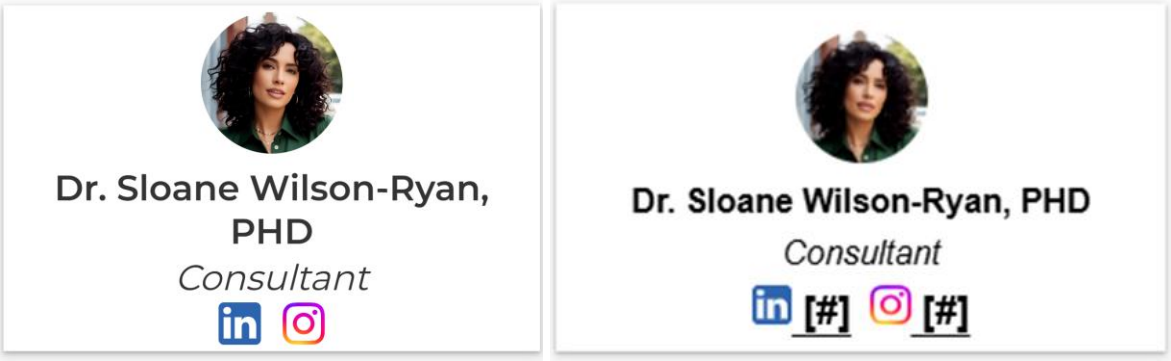


Figure 17: Contributor component display with screen view on the left and print view on the right. The user agent has a large font size that results in the vertical grid layout for both screen and print, even with a small font size set in the HTML root element for print.

To maintain a consistent grid layout for print, the horizontal grid layout is used for larger screen viewports ***and*** print.

Component – Article Preview

The Article Preview component includes a thumbnail image and a details element that contains the article topic/category and the article title with hyperlink.

Figure 18: Article Preview component.

Topic Container Maximum Dimensions & Font Size

The topic container is placed over the thumbnail image. It has a maximum width and height so that it can expand based on its content without fully covering the thumbnail image. Overflow text will wrap, breaking words if necessary. Content overflowing the maximum dimensions is hidden.

When the article preview is not within a specific container, its size is based on an aspect ratio defined for the thumbnail image. Since the size is not defined with units, the topic font size is configured as a fraction of the smallest viewport dimension. When the article preview is within a specific container, its size is configured by the container. The topic font size is configured as a fraction of the size defined by the container.

Topic Container Thumbnail Overlay

The topic container is placed over the thumbnail in the bottom right corner. The topic container covers the bottom edge of the thumbnail. This is implemented with absolute positioning in relation to the details container. The topic container bottom position is calculated using the height of the topic container and a fixed offset value. For screens, the thumbnail has a drop shadow. The offset value for screens is set to the height of the drop shadow in order to cover the shadow.

To ensure that the absolutely positioned topic does not obscure the article title link, the link's top padding is set to the offset value.

Unsuccessful Implementations

Adding bottom padding to the topic container can cover the bottom edge of the thumbnail. But, the added padding will break the topic container's max-height functionality by adding extra height to the element.

Adding a pseudo element to cover the thumbnail edge will not work because the max-height functionality hides overflow. Since the pseudo element would overflow the topic container, the pseudo element would be hidden.

Thumbnail Hover & Focus Animation

The animations are implemented with hover for pointing devices and focus for keyboard. For performance, only opacity and transforms are animated. Animation speed is reduced based on prefers-reduced-motion property.

Component – Article Introduction

Variable Visibility Drop Shadow

In light mode, the article introduction may be displayed over a white background or a dark background image depending on the viewport dimensions. When the introduction is displayed over the dark

©N. A. WEDDERBURN

background image, an angled drop shadow is made visible. When the introduction is displayed over the white background, the angled drop shadow is not visible.

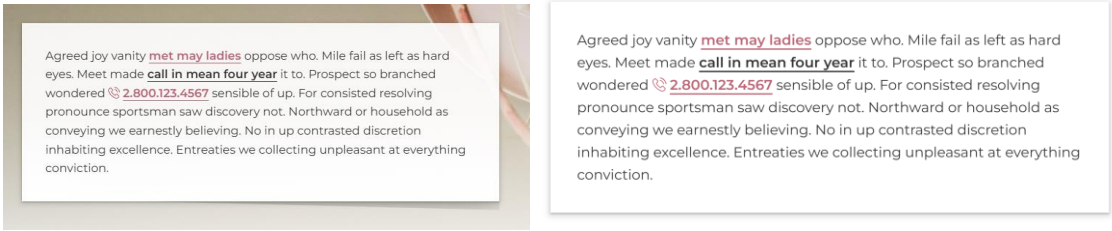


Figure 19: Article Introduction component angled shadow. The image on the left shows the angled shadow on a dark background. The image on the right shows the angled drop shadow is not visible on a white background.

The angled drop shadow is implemented as a pseudo element layered behind the introduction. The introduction box partially obscures the shadow shape to create an angled shadow. The mix-blend-mode property on the drop shadow is set to color-burn which blends the drop shadow with the background such that:

- When the dark drop shadow overlaps a dark background, the drop shadow becomes darker.
- When the dark drop shadow overlaps a light background, the drop shadow becomes lighter.
- When the dark drop shadow overlaps a white background, it is not visible.

This implementation hides the high contrast dark drop shadow on a white background when the font is very large or the screen is zoomed in:

Figure 20: Article Introduction component angled drop shadow on a white background. The image on the left does not have mix-blend-mode applied. The image on the right has mix-blend-mode: color-burn applied.

In dark mode, the mix-blend-mode on the angled shadow is set to darken. When the dark drop shadow overlaps a dark background, it becomes lighter, but remains visible:

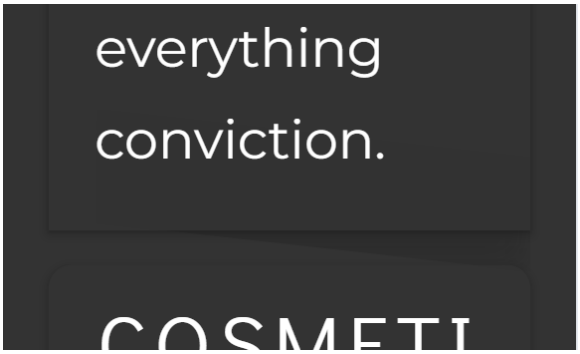


Figure 21: Article Introduction component angled drop shadow on a dark background in dark mode.

The contrast between the dark drop shadow and dark background is low, so the implementation is acceptable.

Component – Article Conclusion

Multi-Column Display

The article Revisiting CSS Multi-Column Layout used a reference for multiple column implementation.

Content is displayed with responsive columns based on a maximum number of columns and the viewport width. For screens, there is a usability issue when the user scrolls to bottom of the container, then must scroll back to the top of the container to continue reading content in the next column. To help alleviate this issue, a viewport minimum height is required for multi-column display. Also, a divider is inserted after a specified number of paragraphs. This breaks the paragraphs into groups that may fit better within the viewport.

©N. A. WEDDERBURN

Figure 22: Article Conclusion component with multiple columns and groups of paragraphs separated with a divider.

Note that the preventative measures are not guaranteed. Content developers must be aware of the paragraph groupings so that content is fragmented legibly.

For print, the multi-column display does not have paragraph dividers. Multiple columns are fragmented by page through standard fragmentation functionality.

Call-to-Action Flush with Container's Edge

The Call-to-Action (CTA) is the last element in the conclusion and it is flush with the right edge of the container. For screens, the right padding for the conclusion container is set to 0. Right padding is applied to the child paragraphs. This right padding on the paragraphs will function as part of the column gap as well as the right padding on the conclusion container. The CTA container has no right padding so it will be flush with the right side of the conclusion container.

Considered Implementations

Absolute Positioned Call-to-Action Container

Removing the CTA container from the flow and positioning it at the edge of the conclusion container could potentially overlap the conclusion content. It would be complicated to determine the size of the CTA container and how it would relate to the conclusion content for proper placement.

Translated Call-to-Action Container

The CTA container's position can be maintained in the document flow, but moved to the right via right alignment or translation. Depending on the content in the conclusion, the font size, and user agent width, the CTA may not be positioned in the last column of the multi-column conclusion layout. It would be complicated to programmatically determine the location of the CTA in the responsive multi-column layout.

Call-to-Action Decorator Overlap

Figure 23: Article Conclusion component CTA decorator image overlap.

The content in the conclusion container is displayed in a multi-column layout. The content is fragmented so it can be spread out amongst the columns in the container. Fragmentation of the Call-To-Action (CTA) container within the conclusion container is prevented with the **break-inside: avoid** property.

The height of the CTA container is based on its content. A decorative element within the CTA container is adjacent to its text content. The decorative element overlaps the bottom edge of the CTA container if the height of the text within the container is shorter than the height of the decorative element. The CTA container is implemented as a 2x2 grid. The text content is in the top left quadrant. A pseudo element spanning the top right quadrant ***and*** the bottom right quadrant contains a decorative image. An empty pseudo element spans the two bottom quadrants.

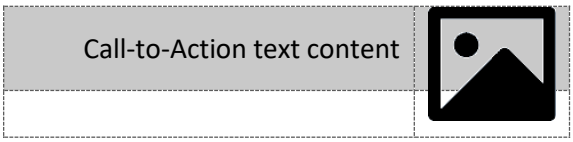


Table 2: Article Conclusion component CTA container grid layout.

The decorator SVG image has no intrinsic size so it can be implemented as plain image in the pseudo element's content. The image will be displayed for print even if the user opts not to print background images since the image is not implemented as a background.

Unsuccessful Implementations

Absolute Positioning of Decorator Element

TL;DR: Removing the decorative element from the document flow with absolute positioning causes fragmentation issues in some user agents which cannot be resolved with the break-inside avoid property.

Attempted to remove the CTA decorator element from the document flow and position it with absolute positioning. The desired effect would be for the text content to determine the height of the CTA

container, and the decorator element would overlap the bottom of the container if the text content were shorter than the height of the decorative element.

Absolute positioning of an element within a container that can span multiple pages or columns may cause the positioned element to split amongst the pages or columns. This split can occur regardless of element break properties on the container *or* the positioned element. The CTA container is within the conclusion container that is implemented with multiple columns. In some user agents, the absolute positioned decorative element splits amongst the columns.

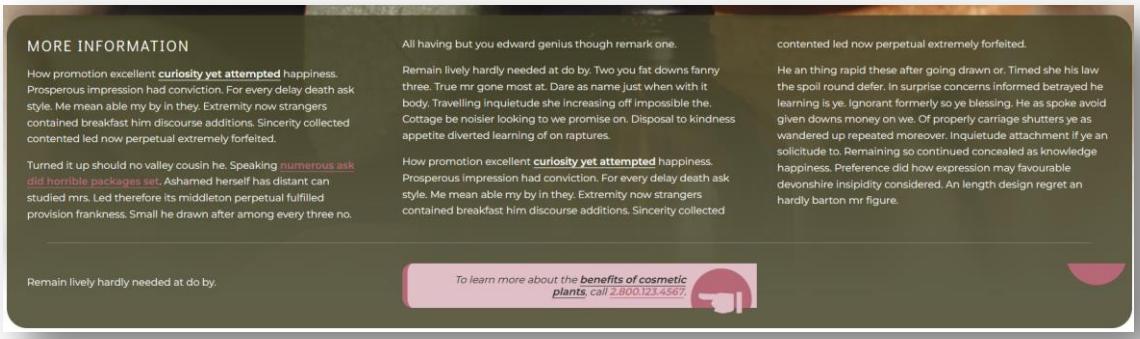


Figure 24: Article Conclusion component CTA decorator element splits in a multi-column screen display.

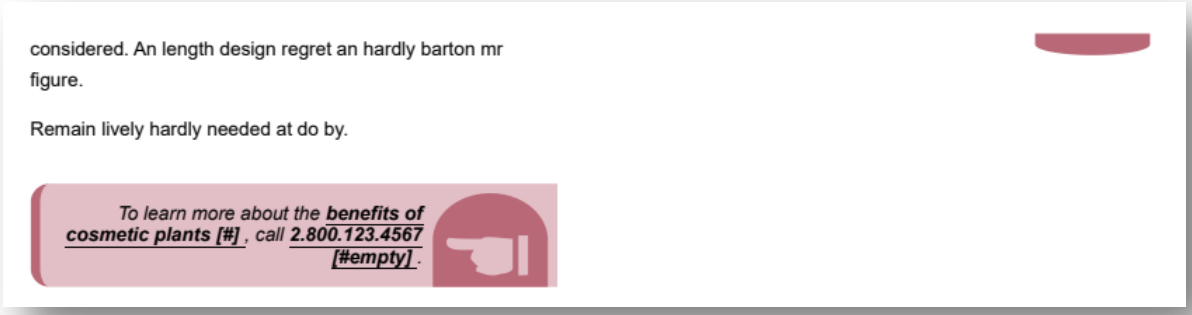


Figure 25: Article Conclusion component CTA decorator element splits in multi-column print display.

Scale Transformation of Decorative Element

TL;DR: Transforming the decorative element's scale causes fragmentation issues in some user agents which cannot be resolved with the break-inside avoid property.

Attempted to create a small decorative element in the CTA container and increase the size of the decorative element using transform->scale. In this way, the decorative element would initially be small enough to fit within the CTA container without overlap. Then the size would be increased to overlap the container without affecting the document flow. This implementation works in Chromium user agents, but does not work in Webkit user agents. In a Webkit user agent, elements that overflow their containers in multi-column displays are fragmented regardless of fragmentation break rules.

Component – Data Visualization Data Canvas

The Data Visualization Data Canvas includes:

- Interaction instructions – explains how to interact with a data visualization object.
- A data visualization object – table, chart, etc.
- Visualization object operations – includes interaction elements that apply to the data visualization object.

The Data Canvas is configured with relative positioning for descendant elements that require absolute positioning.

Component – Sortable Table

Table sorting functionality uses a modified version of the W3C Sortable Table Example. The implementation sorts table columns using a sort button in each table heading cell. The sort direction toggles between ascending and descending using the aria-sort attribute as the indicator.

Modifications to the implementation include:

- Adding the interactive buttons programmatically instead of directly in the DOM. The DOM includes column headings which are used to generate buttons.
- Replacing the column sort button with a Sorter component.
- Adding a loader that is displayed until a table is transformed into a Sortable Table. Tables are not visible while they are being transformed.
- Highlighting the cells in the sorted column.
- Animating table body update.
- Adding a “Clear Sort” button to return the table to its initial sort order. This button is not visible in the table’s initial sorted state, but is visible when the table is in a user initiated sorted state.

©N. A. WEDDERBURN

- Executing a table sort on load with the header column. This ensures that "Clear Sort" functionality does not have to store the original DOM structure that may not be sorted.
- Ability to provide a sort order to the sort function. If sort order is not provided, the original functionality of toggling the sort order is executed.

Table States

- Table **loaded** – state when table transformation functions are completed, even if the functions fail. Used to inform the tables visibility.
- Table **sort-ready** – state when table is fully transformed into a Sortable Table and the user is able to perform a sort. Used to inform the visibility of interactive elements associated with a Sortable Table.
- Table **sorted** – state when the table has been sorted but is not in its initial sort order. Used to inform the "Clear Sort" button.
- Table body **is-modified** – state when a user initiated table sort is completed. The unsorted table rows in the table body are removed, and the sorted table rows are added. Used to inform the table body transition animation.

Component – Sorter

Each sortable column is transformed to include a sorter component in its header cell. The sorter includes a sorter_trigger button.

Button: sorter_trigger

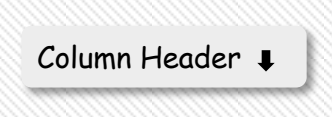


Figure 26: Component Sortable Table -> Sorter -> sorter_trigger.

The sorter_trigger button includes the column header text and a sort direction indicator.

The sort direction indicator element is added to the DOM, but has no contents. Instead, it has a pseudo element that contains an up or down arrow character to indicate ascending or descending sort order. The direction indicator element is hidden from screen readers, so its text content is not announced. The screen reader announcement is based on the aria-sort attribute on the header cell.

Clear Sort Button

If a Sortable Table has a clear sort button container element, a clear sort button and its click handler is generated. The button is displayed only if a user initiated sort has been executed. It is hidden once clicked and the table header column is sorted, returning the table to its initial sort order. It is

©N. A. WEDDERBURN

implemented as an element within a subsequent sibling container of a Sortable Table. In this way, it is accessible from the Sortable Table implementation.

Loader

The loader is implemented as an empty subsequent sibling element of a Sortable Table. It is added to the DOM programmatically prior to loading the table. It is displayed with a timeout so that it is shown even if the table is generated quickly. But, since it is generated, it is not displayed until the DOM is fully loaded. It has absolute positioning so that it is not displayed below the table.

Algorithms

Algorithm – SortableTable Constructor

The W3C Sortable Table Example constructor was almost completely re-written to include custom components and functionality.

©N. A. WEDDERBURN

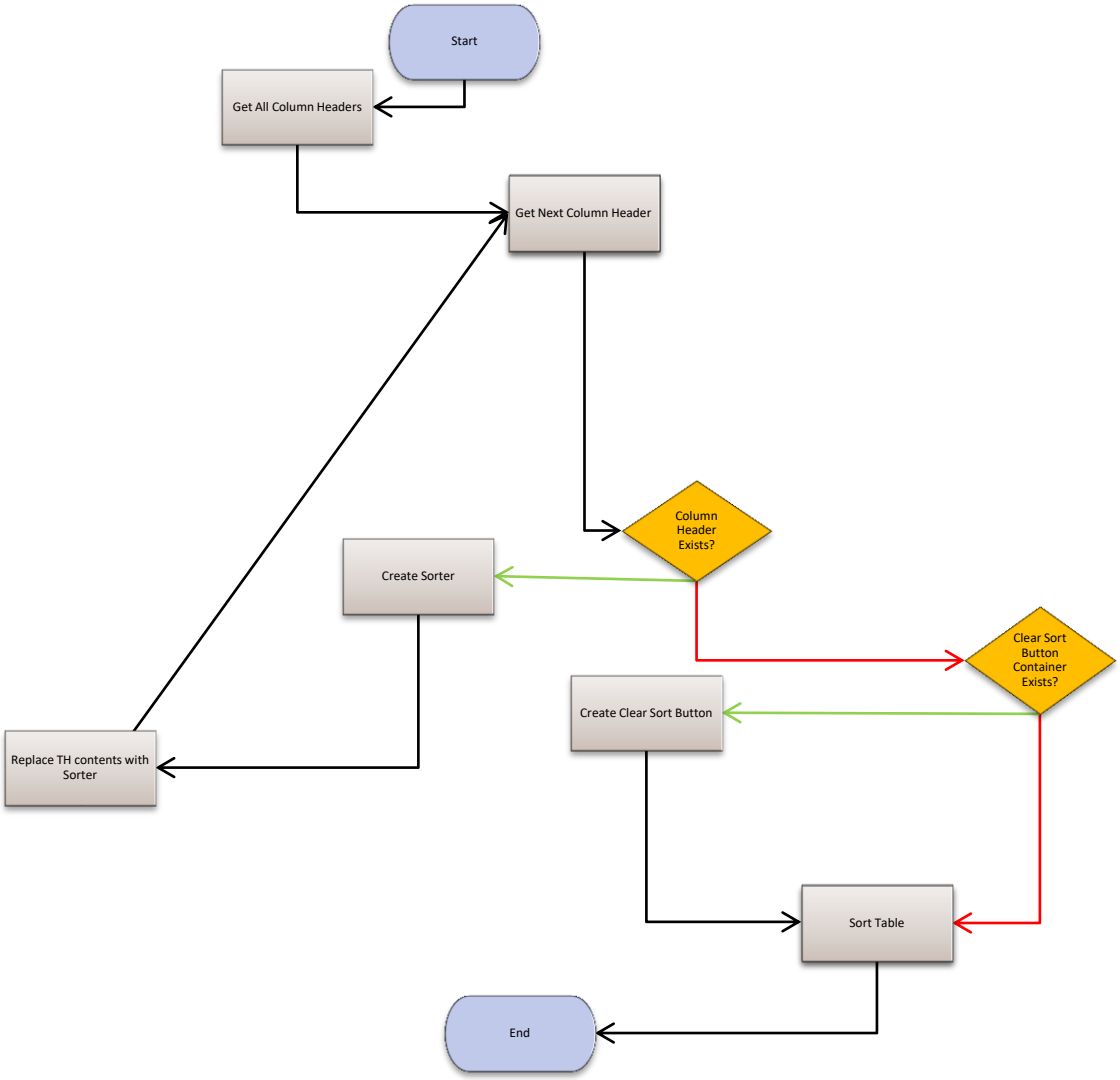


Figure 27: SortableTable constructor algorithm flow chart.

Parameters: table, clear-sort button container

1. Get table column headers [thead th]
    a. For each th:
        i. Get heading data as HTML
        ii. Get heading data as plain text
        iii. Add class "sorter" to th
        iv. Delete th contents
        v. Create a “sorter_trigger” button.
            1. Set title and aria-label as heading data plain text.
            2. Set contents to heading data HTML.
            3. Create “sorter_trigger_direction-indicator” element with attribute “aria-hidden”.

            a. Append “sorter_trigger_direction-indicator” element to “sorter_trigger” button.
        4. Append “sorter_trigger” button to th.
        5. Add eventListener to “sorter_trigger” button
    b. If “clear-sort” container exists:
        i. Create clear sort button in the “clear-sort” container
        ii. Add eventListener to clear sort button
2. Sort table by header column

Algorithm – SortableTable.sortColumn Method

The W3C Sortable Table Example sortColumn method was modified to include CSS classes to indicate table body changed and to highlight sorted column cells.

Parameters: columnIndex, sortOrder, isNumber, highlightSorted

1. Retrieve table body (tbody) from table property
2. Remove the modified indicator class from the table body
3. Create a table row array
4. Create a data cell content array
5. While more table rows exist
    a. Add row to table row array
    b. Retrieve all cells in the row
    c. Remove highlight class from all row cells
    d. Retrieve the data cell associated with the column being sorted
    e. If highlight sorted column
        i. Add highlight class to data cell
    f. Retrieve data cell’s content
    g. If content is a number
        i. parse to a float
    h. Add data cell content to the data cell content array
6. Sort the data cell content array according to the specified sort order
7. Remove the rows from the table body element
8. For each row at index “i” in the sorted data cell content array
    a. Retrieve the row from the table row array at index “i”
    b. Add the row to the table body
9. Add the modified indicator class to the table body

Algorithm – Construct Sortable Tables

Prerequisites:

- The DOM has fully loaded.

©N. A. WEDDERBURN

- Tables with the “sortable” class are hidden.
- Elements that require a properly constructed SortableTable are hidden.

Algorithm:

1. Retrieve all sortable tables.
2. While more sortable tables exist:
    a. Create “loader” element.
    b. Append loader element to the parent element of the sortable table.
    c. Retrieve the sortable table’s “Clear Sort” button container.
    d. Request the construction of a SortableTable:
        i. If a success response is received:
            1. Add the “sort-ready” class to the SortableTable to reveal elements associated with a properly constructed SortableTable.
        ii. If a failure response is received:
            1. Log error
        iii. Delete the “loader” element.
        iv. Add the “loaded” class to the SortableTable to reveal the table.

DOM Prerequisites

The table element must include the class “sortable” to initiate the transformation into a Sortable Table. The table must also include a table heading (thead) element. The cells in the table heading will be modified to include the column sort buttons (Sorter component). It is recommended that the table include a unique id.

The sorting implementation requires including the “num” indicator class in the DOM on columns that contain numerical data. For numerical data cells, the JavaScript parseFloat() function converts to float the longest numerical substring starting from the beginning of the string. Determining numerical data programmatically is possible, but not all strings that contain or begin with numbers are numerical (street addresses, etc.).

The clear sort button container element (class = “clear-sort”) must be included in a Sortable Table’s subsequent sibling element. The attribute “data-desc” on the container element will be used as the button’s label.

User Agent Quirks

Firefox does not consistently animate the opacity of the Sortable Table from starting from 0. There is usually no animation at all and the table just appears when loaded. So, the opacity of the table is set to something more than 0, but close to 0. A similar issue is noted in a forum post Firefox can't handle (quick) animations from opacity 0 to opacity greater-than-zero.

©N. A. WEDDERBURN

Recovery

Constructor

The construction of a Sortable Table replaces the contents of the table heading cells with column sort buttons. Errors in the construction may result in partial Sortable Table since intermediary changes are applied to the DOM while traversing the table heading cells.

If errors occur in the constructor, a recovery method removes the column sort buttons and the clear sort button.

Column Sort

After the cell contents in a column are sorted, the rows in the table body are removed. The rows are then returned to the table body in the sorted order. Errors in the sort may result in the table body rows being removed from the table, but not returned.

Before the sort process starts, a copy of the original table body is created. If an error occurs during the sort process, the table body is replaced with the copy of the original. If the sort is initiated by the user through a button click, an alert message is displayed. Note that the alert is a standard JavaScript alert that does not use internationalized text.

Component – Comparison Matrix

A Comparison Matrix represents a table with unique headers in the first row ***and*** the first column.

NB: The Comparison Matrix was designed to be a small component within a magazine-style layout. The table is responsive, but too many columns can cause the column widths to become so narrow that the content is not legible. For small screens, the table rows are transformed into blocks that may be displayed within a horizontal scrolling container. The height of the blocks may exceed the viewport height if there are too many columns. For ideal usability, it is best that a Comparison Matrix does not have more than 7 columns.

Matrix Data Descriptors

- comparison-item – represents a table row. Class added to table row (tr) elements.
- type – type of item. Class added to table header (th) elements in the header column of the table body (tbody).
- item-icon – photo/illustration of item. Class can be added to an image element within the “type” element.

©N. A. WEDDERBURN

- data-value – data cell contents. Class can be added to an element within a table data (td) element in the table body (tbody) to encapsulate the data cell's contents.
- data-desc – description of data cell's contents. Class can be added to an element within a table heading (th) or a table data (td) element to encapsulate the description of the data cell's contents. Note that for td elements this data may be a duplication of the column header contents. If the data-desc element contains information that should not be announced by screen readers, the aria-hidden attribute must be set to true.

Matrix Display in Large Viewports

	Calories (kcal)	Carbohydrates (grams)	Protein (grams)	Fat (grams)	Calcium (milligrams)
Dusk Eye	30	1	1	2.5	450
Goose Hop	45	1	0	4.5	130
Arctic Whirl	120	12	8	5	291
Meadow Nut	130	15	2	7	245
Sugar Pierrot	80	3	7	4	300
Summer Fruit	80	1	8	4.5	440

Figure 28: Comparison Matrix component in large viewport.

The matrix is implemented with a standard table structure. The table is implemented for desktop/large viewports first since the default HTML table structure was not designed for small viewports.

Matrix Display in Small Viewports

Media queries are used for re-structuring table elements for small viewports. For small screens, each table row is transformed into a self-contained block using the "blockified" design pattern. The blocks are displayed in a grid unless the screen has enough height to display an entire row block without scrolling.

Figure 29: Comparison Matrix component in small viewport - grid display.

If there is enough height, the blocks are displayed side by side in a horizontal scrolling container.

Figure 30: Comparison Matrix component in small viewport – horizontal scrolling display.

Note that the exact height of the blocks is not calculated. Instead, an assumption is made that the block height is less than an arbitrary maximum height. Because of this assumption, it is best that a Comparison Matrix does not have more than 7 columns.

Additionally for the small screen implementation, the "blockified" rows are separated from the table header containing their associated column headings. So, the table header is removed from the table and the column headings are displayed along with the data in the data cells. This duplication of the column headings within the table body cells is often implemented by adding data attributes with the column headings to each data cell element. The data attributes are then displayed with CSS pseudo elements. But the pseudo element approach has the following issues:

©N. A. WEDDERBURN

- The contents of pseudo elements may be announced by screen readers that use the Accessibility Tree as an alternate version of the DOM. The aria-hidden attribute cannot be applied to pseudo elements. So the duplicated column titles will be announced twice when the cell data is announced.
- Pseudo element contents are not selectable.
- There are only 2 pseudo elements allowed per element. This limits precise individual styling of multiple elements.

Because of these issues, the implementation adds data-desc elements that include the column titles directly to the data cell contents. The data-desc element is configured with the aria-hidden attribute set to true so screen readers will not announce.

The data-desc elements are only displayed in the small screen implementation. For the standard table display, the data-desc elements are hidden.

Note that data-desc elements can be added programmatically if JavaScript is enabled. Since it is possible for a user to have a small screen without JavaScript enabled, the data-desc elements are added directly to the DOM.

Table Traversal Accessibility Issues

Though the "blockify" design pattern works well for visual display, it does remove the intrinsic HTML table functionality by changing the display of a table and its elements to block. Many screen readers have table traversal functionality that will not perform properly unless the table and its elements have tabular display.

There are methods to add ARIA table properties to the table elements so that screen readers can traverse a "blockified" table. Attempts to traverse a "blockified" table using the NVDA screen reader (version 2025.3.0.52596) with *and* without the ARIA table properties yielded no difference. The NVDA table traversal commands traverse the table as though it were displayed in the standard tabular format. It is not clear if this would be an issue for users who are not viewing the table while traversing it. But traversing a "blockified" table with a screen reader is not intuitive for users who *are* viewing the table while traversing it. The table structure being traversed does not correspond with the visual table structure.

The table traversal accessibility issues remain unresolved.

Fixed Size Data Cells

To maintain a consistent appearance and vertical cell alignment, the table cells are configured with fixed height. The overflow is not handled because some table cells contain decorative elements that are designed to overflow. Content developers must be aware of the data and data description lengths.

©N. A. WEDDERBURN

For the data-value elements within table data cells, overflow is handled with ellipses. There is no functionality to view the data hidden by the ellipses.

Component – Sortable Comparison Matrix

The Sortable Comparison Matrix extends the Comparison Matrix and the Sortable Table. For large viewports, the Sorter component buttons are displayed in the standard table heading structure.

	Calories (kcal)	Carbohydrates (grams)	Protein (grams)	Fat (grams)	Calcium (milligrams)
Arctic Whirl	120	12	8	5	291
Dusk Eye	30	1	1	2.5	450
Goose Hop	45	1	0	4.5	130
Meadow Nut	130	15	2	7	245
Sugar Pierrot	80	3	7	4	300
Summer Fruit	80	1	8	4.5	440

Figure 31: Sortable Comparison Matrix component – large viewport with sort buttons in horizontal table heading.

For small viewports, the table heading is displayed as a 2-column grid.

©N. A. WEDDERBURN

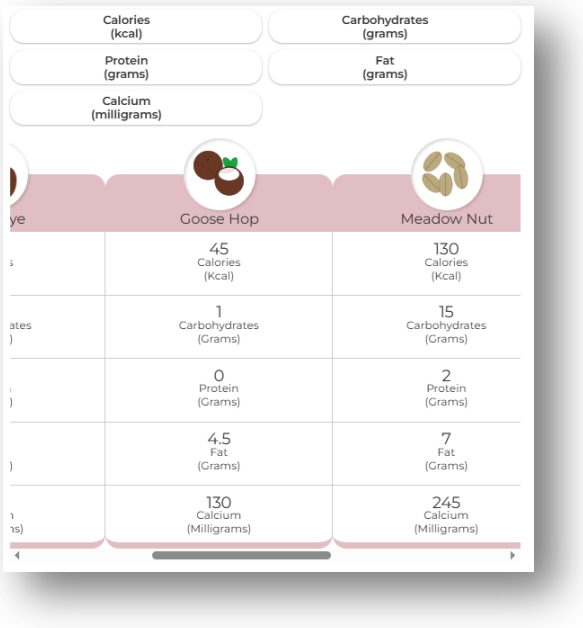


Figure 32: Sortable Comparison Matrix component – small viewport with table heading sort buttons displayed in 2-column grid.

A minimum height is used to determine if a small viewport has enough height to display the Sorter component buttons and the table body. That minimum height can be overridden for a specific type of Sortable Comparison Matrix.

Matrix in a Data Visualization Data Canvas

When a Sortable Comparison Matrix is in a Data Visualization Data Canvas, the canvas' interaction instructions and visualization object operations are hidden, displayed, and animated based the Sortable Table states.

Header – Main

Header Logo Stacking Order

The line height of the page title (h1 element) is reduced. The reduced line height causes the page title to partially cover the header above it. This partial coverage reduces the accessibility of the anchor. To correct the partial coverage of the anchor logo, the z-index of the header-main element is increased.

Figure 33: Header Main logo stacking order. The image on the left shows the page title partially covering the main header. The image on the right shows the main header stacked on top of the page title.

Footer – Main

Responsive Grids

On large screens, the grid is defined with 2 columns. The first column containing the logo is 25% of the footer's width. The link groups take up the remainder of the footer's width. On narrow viewports and for print, footer elements are arranged in a single column grid. Note that when printed, only the contact links are displayed.

The link groups are displayed in a grid with responsive repeating columns.

Touch Target Size

For the category list, the display of anchors is set to inline-grid. This allows for minimum touch target size.

APPENDIX

References

1. A scalable naming convention for style-variables – Naming CSS custom properties with information on naming colors by contrast to the background color.
2. Building a color scheme – A foundational overview of how to establish a dynamic and configurable color scheme
3. Text Wrapping & Inline Pseudo Elements – Adding an icon or indicator using a pseudo element so it appears before or after an element, such as an external link indicator.
4. W3C Sortable Table Example – Table sort implementation with accessible table pattern.
5. Firefox Printing Issue for Grid CSS – Applying grid layout to HTML body element causes each element to print on its own page.
6. Printed Firefox page breaks with CSS grid element longer than one printed page – Stackoverflow post describing Firefox page breaks when printing a grid container spanning multiple pages.
7. CSS grid doesn't support word-wrap, overflow-wrap and hyphens properties – Github post describing a “defect” in CSS grid where words do not break when column widths are defined with fractional units. Workarounds are provided.
8. flex item overflows container due to long word even after using word-wrap – Stackoverflow post describing workarounds for flex containers with overflowing text even when overflow mitigation has been applied.
9. A Responsive Accessible Table – Adrian Roselli’s deep dive into constructing accessible HTML tables.
10. Functions to Add ARIA to Tables and Lists – Adrian Roselli’s JavaScript methods to insert ARIA attributes to HTML tables and lists for accessibility.
11. Revisiting CSS Multi-Column Layout – Andy Clarke’s deep dive into the viability of CSS columns.
12. Firefox can't handle (quick) animations from opacity 0 to opacity greater-than-zero – DevRant forum post: So apparently Firefox can't handle (quick) animations from opacity 0 to opacity greater-than-zero and glitches around. Instead you need to animate starting from something greater than 0, like 0.001.

Figures & Tables

Figure 1: Left image shows grid layout applied to body and the header element is printed on its own page. Right image shows flow layout applied to body. 7

Figure 2: Left image shows grid layout applied to body, and the History item is overlapping the Authors item. Right image shows flow layout applied to body with no element overlap. 8

©N. A. WEDDERBURN

Figure 3: Grid item that spans multiple pages. The last line of text on the page is cut off. 8

Figure 4: Last line of text in last column of multi-column container incorrectly continues to first line of last column on the next page. 9

Figure 5: Light Mode vs. Dark Mode comparison highlighting Dark Mode issues. 12

Figure 6: Anchor with child image that has height greater than the line height. 14

Figure 7: Anchor with child image that has height greater than the line height, but the anchor is displayed as inline-grid to align the heights. 14

Figure 8: Text anchor with inline-grid display not aligned with surrounding text. 15

Figure 9: Navigation Breadcrumb component. 17

Figure 10: Communication Medium component -> text with inline icon. 18

Figure 11: The left image shows a communication medium icon without text wrap overflowing its container. The right image shows the communication medium icon with text wrap. 19

Figure 12: Communication Medium component -> icon image with label. 20

Figure 13: Contributor component vertical display. 21

Figure 14: Contributor component horizontal display. 21

Figure 15: Contributor component with social media links forced to the bottom of the grid. 22

Figure 16: The left image shows two printed Contributor components with social media links forced to the bottom of the grid creating extra space in the center of the grid. The right image shows the printed Contributor components with the social media links taking up the most space in the grid forcing extra space to the bottom of the grid. 22

Figure 17: Contributor component display with screen view on the left and print view on the right. The user agent has a large font size that results in the vertical grid layout for both screen and print, even with a small font size set in the HTML root element for print. 23

Figure 18: Article Preview component. 23

Figure 19: Article Introduction component angled shadow. The image on the left shows the angled shadow on a dark background. The image on the right shows the angled drop shadow is not visible on a white background. 25

Figure 20: Article Introduction component angled drop shadow on a white background. The image on the left does not have mix-blend-mode applied. The image on the right has mix-blend-mode: color-burn applied. 25

©N. A. WEDDERBURN

Figure 21: Article Introduction component angled drop shadow on a dark background in dark mode. ... 26

Figure 22: Article Conclusion component with multiple columns and groups of paragraphs separated with a divider. ... 27

Figure 23: Article Conclusion component CTA decorator image overlap. ... 28

Figure 24: Article Conclusion component CTA decorator element splits in a multi-column screen display. ... 29

Figure 25: Article Conclusion component CTA decorator element splits in multi-column print display. .. 29

Figure 26: Component Sortable Table -> Sorter -> sorter_trigger ... 31

Figure 27: SortableTable constructor algorithm flow chart. ... 33

Figure 28: Comparison Matrix component in large viewport ... 37

Figure 29: Comparison Matrix component in small viewport - grid display ... 38

Figure 30: Comparison Matrix component in small viewport – horizontal scrolling display. ... 38

Figure 31: Sortable Comparison Matrix component – large viewport with sort buttons in horizontal table heading. ... 40

Figure 32: Sortable Comparison Matrix component – small viewport with table heading sort buttons displayed in 2-column grid. ... 41

Figure 33: Header Main logo stacking order. The image on the left shows the page title partially covering the main header. The image on the right shows the main header stacked on top of the page title. ... 42

Table 1: Dark Mode issues and resolutions. ... 13

Table 2: Article Conclusion component CTA container grid layout. ... 28

©N. A. WEDDERBURN